Python Decorators
Student Handout

From simple decorators to real-world web frameworks and routing

1. What is a Decorator?

A decorator is a function that takes another function, adds or changes behavior, and returns a function. It lets us add behavior without rewriting the original function.

Easy mental model:

Decorator = a wrapper around a function.

```
def decorator(func):
    def wrapper():
        print("Before")
        func()
        print("After")
    return wrapper

def hello():
    print("Hello")

hello = decorator(hello)
hello()
```

Output:

```
Before
Hello
After
```

2. The @ Syntax

Python gives us a shorter way to apply a decorator. Instead of writing function = decorator(function), put @decorator directly above the function.

```
def decorator(func):
    def wrapper():
        print("Before")
        func()
        print("After")
    return wrapper

@decorator
def hello():
    print("Hello")

hello()
```

These two forms mean essentially the same thing:

```
@decorator
def hello():
    print("Hello")

# Same idea as:
def hello():
    print("Hello")

hello = decorator(hello)
```

3. Why Do We Use Decorators?

- Add common behavior to many functions.
- Avoid repeating the same code.
- Keep the original function focused on its main job.
- Control access, logging, timing, caching, validation, and permissions.
- Frameworks use decorators to attach meaning to functions, especially routes and handlers.

4. A Realistic Example: Logging

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Finished {func.__name__}")
        return result
    return wrapper

@log_call
def add(a, b):
    return a + b

print(add(3, 4))
```

The add() function still focuses only on addition. The decorator adds logging around it.

5. Decorators with Arguments

The wrapper can accept *args and **kwargs so the decorator works with functions having different arguments.

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print("Function:", func.__name__)
        print("Arguments:", args, kwargs)
        return func(*args, **kwargs)
    return wrapper

@log_call
def greet(name, message="Hello"):
    return f"{message}, {name}"

print(greet("Ali"))
print(greet("Sara", message="Welcome"))
```

*args collects positional arguments, while **kwargs collects keyword arguments.

6. Returning a Value from the Decorated Function

```
def double_result(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return result * 2
    return wrapper

@double_result
def square(n):
    return n * n

print(square(5))  # 50
```

The decorator can inspect, modify, or completely replace the result returned by the original function.

7. A Practical Decorator: Timing a Function

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()

        print(f"{func.__name__}: {end - start:.4f} seconds")
        return result

    return wrapper

@timer
def slow_task():
    time.sleep(1)
    return "Done"

print(slow_task())
```

Timing decorators are useful when profiling slow functions, API calls, database work, or experiments.

8. A Practical Decorator: Authentication / Permission

```
def requires_admin(func):
    def wrapper(user, *args, **kwargs):
        if user != "admin":
            return "Access denied"
        return func(user, *args, **kwargs)
    return wrapper

@requires_admin
def delete_database(user):
    return "Database deleted"

print(delete_database("admin"))  # Database deleted
print(delete_database("Ali"))    # Access denied
```

Web applications often use this pattern to protect routes or operations based on authentication and permissions.

9. A Practical Decorator: Caching with functools

```
from functools import lru_cache

@lru_cache
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(40))
```

@lru_cache stores previous results so repeated calls can be much faster. This is an example of a decorator supplied by Python's standard library.

10. A Practical Decorator: Keeping Function Metadata

```
from functools import wraps

def logger(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("Calling", func.__name__)
        return func(*args, **kwargs)
    return wrapper

@logger
def greet(name):
    """Return a greeting."""
    return f"Hello, {name}"

print(greet.__name__)  # greet
print(greet.__doc__)   # Return a greeting.
```

functools.wraps preserves useful metadata such as the original function's name and docstring. It is good practice when writing decorators.

11. Decorators that Take Their Own Arguments

Sometimes we want a decorator itself to receive settings, such as a retry count or permission name. This creates three layers: decorator settings → decorator → wrapper.

```
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            result = None
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def hello():
    print("Hello")
```

```
hello()
```

This is why decorator factories often have three nested functions.

12. Python's Built-in Decorators

```
class Student:
    school = "UoB"

    def __init__(self, name):
        self.name = name

    @classmethod
    def from_uppercase(cls, name):
        return cls(name.upper())

    @staticmethod
    def is_adult(age):
        return age >= 18

    @property
    def display_name(self):
        return self.name.title()
```

@property, @staticmethod, and @classmethod are built-in decorator-based tools in Python. You will see them often in object-oriented code.

13. FastAPI: Decorators Used for Routing

FastAPI uses decorators to connect a Python function to an HTTP route. The decorator tells FastAPI what URL path and HTTP method should trigger the function.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello"}

@app.get("/students")
def get_students():
    return {"students": ["Ali", "Sara"]}

@app.post("/students")
def create_student():
    return {"message": "Student created"}
```

Here, @app.get('/') means: register home() as the handler for an HTTP GET request to '/'. @app.post('/students') similarly registers create_student() for POST requests.

This is a major real-world use of decorators: the decorator is not just adding print statements. It is registering the function with a framework.

14. FastAPI Route Parameters

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/students/{student_id}")
def get_student(student_id: int):
    return {"student_id": student_id}
```

The decorator defines the route pattern, while the function handles the request. FastAPI can then use the parameter information to process requests and validate types.

15. FastAPI Authentication / Dependencies

```
from fastapi import FastAPI, Depends

app = FastAPI()

def get_current_user():
    # Real code would verify a token/session.
    return {"username": "Ali"}

@app.get("/profile")
def profile(user = Depends(get_current_user)):
    return user
```

FastAPI also uses dependency declarations. Depending on the specific feature, the syntax may combine decorators with dependency functions to express authentication, database access, or reusable request logic.

16. Flask: Routing with Decorators

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello Flask"

@app.route("/about")
def about():
    return "About page"

if __name__ == "__main__":
    app.run()
```

Flask uses decorators in a very similar way: they register functions as route handlers.

17. Django: Decorators for Access Control

```
from django.contrib.auth.decorators import login_required

@login_required
```

```
def dashboard(request):
    return render(request, "dashboard.html")
```

Django provides decorators that can protect views. In this example, users normally need to be logged in before the view can be used.

18. Why Frameworks Love Decorators

Frameworks often need to collect information about functions: which URL they belong to, which HTTP method they use, whether authentication is required, whether a task should run asynchronously, or what dependencies it needs.

A decorator provides a clean way to say:

Function + extra framework meaning

19. Decorator vs Normal Function Call

Pattern	Idea	Example
Normal call	Call a function directly.	add(2, 3)
Function as argument	Pass one function into another.	decorator(add)
Decorator	Wrap/modify/register a function.	@timer
Decorator factory	Configure a decorator before applying it.	@repeat(3)

20. Decorators + Closures: The Connection

Many decorators are built using closures. The inner wrapper function remembers the original function (func) from the enclosing decorator function.

```
def decorator(func):
    def wrapper():
        print("Before")
        func()       # wrapper remembers func
        print("After")
    return wrapper
```

So decorators are an excellent place to see closures in real code: the wrapper keeps access to func after decorator() has returned.

21. Stacked Decorators

```
def first(func):
    def wrapper():
        print("First")
        func()
    return wrapper

def second(func):
    def wrapper():
        print("Second")
        func()
    return wrapper
```

```
@first
@second
def hello():
    print("Hello")

hello()
```

Multiple decorators can be stacked. Python applies the lower decorator first, then the next one around it. The call therefore passes through multiple wrappers.

22. Common Mistakes

Forgetting to return the wrapper: The decorated function may become None.

Forgetting to call func(): The original function may never run.

Not returning the original result: A decorated function that used to return a value may unexpectedly return None.

Not using *args and **kwargs: The decorator may only work with one very specific function signature.

Skipping functools.wraps: Function name/docstring and other metadata can be replaced by the wrapper's metadata.

23. When You Will Actually See Decorators

- FastAPI and Flask route definitions.
- Django authentication and permission checks.
- Logging and monitoring.
- Performance measurement and profiling.
- Caching and memoization.
- Retries around unreliable operations.
- Validation and input checks.
- Authorization and role-based access control.
- Task/job systems and framework registration.
- Python built-ins such as @property, @classmethod, and @staticmethod.

24. A Useful Real-World Pattern: Retry

```
import time
from functools import wraps

def retry(times):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last_error = None

            for _ in range(times):
                try:
                    return func(*args, **kwargs)
                except Exception as error:
                    last_error = error
                    time.sleep(0.5)
```

```
            raise last_error

        return wrapper
    return decorator

@retry(3)
def fetch_data():
    # Imagine an API call here.
    raise ConnectionError("Temporary failure")
```

This pattern can be useful for network/API operations, although production systems often use a dedicated retry library or framework feature for more advanced retry policies.

25. Practice Exercises

1. Write a @bold decorator that prints a message before and after a function.

2. Write a @timer decorator that reports execution time.

3. Write a decorator that checks whether the first argument is positive.

4. Write a @repeat(times) decorator that calls a function multiple times.

5. Create your own @admin_only decorator for a simple user role.

6. Write a Flask or FastAPI route using a decorator and explain what the decorator registers.

7. Modify one of your decorators to use functools.wraps.

8. Explain how a decorator can use a closure to remember the original function.

26. One-Minute Summary

- Decorator = a function that works with another function to add behavior or register it.
- @decorator is shorthand for replacing a function with the decorated version.
- A wrapper function is commonly used inside a decorator.
- *args and **kwargs make wrappers flexible.
- functools.wraps preserves the original function's metadata.
- Many decorators use closures because the wrapper remembers the original function.
- FastAPI and Flask use decorators heavily for routing.
- Django uses decorators for things such as authentication/access control.
- Decorators are useful far beyond logging: caching, timing, validation, permissions, retries, and framework registration.

Mental model: Decorator = take a function, wrap it or register it, and give it extra behavior/meaning.